

Práctica 1: Optimización de Código

Asignatura Tópicos A

Nombre: Luis Antonio Muñoz Vásquez

Universidad de Magallanes

Mayo de 2026

Facultad de Ingeniería

Departamento de Ingeniería en Computación

Punta Arenas, Chile

Índice

1. Introducción

La presente práctica tiene como finalidad estudiar y comparar el rendimiento de distintas implementaciones de operaciones de álgebra lineal, considerando el uso de Octave, lenguaje C y funciones de la biblioteca CBLAS.

Las operaciones analizadas corresponden al producto punto, la multiplicación matriz-vector y la multiplicación matriz-matriz. Estas operaciones permiten observar diferencias importantes entre código interpretado, código compilado, código optimizado y rutinas especializadas de álgebra lineal.

Según el enunciado de la práctica, se debe comparar el tiempo de ejecución entre operadores de Octave y funciones CBLAS. Además, se deben implementar las operaciones en lenguaje C, aplicar optimización de código y comparar sus resultados con CBLAS. En total, se solicita generar seis gráficos asociados a los tiempos de ejecución de los algoritmos implementados.

2. Objetivos

2.1. Objetivo general

Comparar el rendimiento de distintas implementaciones de operaciones de álgebra lineal, considerando Octave, C manual, C optimizado y CBLAS.

2.2. Objetivos específicos

- Implementar y medir el tiempo de ejecución del producto punto, multiplicación matriz-vector y multiplicación matriz-matriz.
- Comparar los operadores de Octave con las funciones CBLAS correspondientes.
- Implementar versiones manuales en C para cada operación.
- Aplicar técnicas de optimización de código y evaluar su impacto en los tiempos de ejecución.
- Analizar los resultados obtenidos en función de la complejidad algorítmica de cada operación.

3. Marco teórico

3.1. Complejidad algorítmica

La complejidad algorítmica permite estimar el crecimiento del costo computacional de un algoritmo en función del tamaño del problema. Para esta práctica se consideran tres operaciones principales.

El producto punto entre dos vectores de tamaño n requiere recorrer ambos vectores una vez, por lo que su complejidad es:

$$O(n) \tag{1}$$

La multiplicación matriz-vector, considerando una matriz de tamaño $n \times n$ y un vector de tamaño n , requiere dos ciclos anidados, por lo que su complejidad es:

$$O(n^2) \tag{2}$$

La multiplicación matriz-matriz, considerando dos matrices cuadradas de tamaño $n \times n$, requiere tres ciclos anidados, por lo que su complejidad es:

$$O(n^3) \tag{3}$$

3.2. Octave

GNU Octave permite realizar operaciones de álgebra lineal mediante operadores nativos, tales como el producto punto, la multiplicación matriz-vector y la multiplicación matriz-matriz. Estos operadores suelen estar implementados mediante rutinas optimizadas y compiladas, por lo que presentan mejores tiempos que una función interpretada escrita directamente en Octave.

3.3. CBLAS

CBLAS corresponde a una interfaz en lenguaje C para utilizar rutinas BLAS. En esta práctica se utilizaron las funciones:

- `cblas_ddot`: producto punto.
- `cblas_dgemv`: multiplicación matriz-vector.
- `cblas_dgemm`: multiplicación matriz-matriz.

3.4. Optimización de código

En esta práctica se aplicaron técnicas de optimización sobre las implementaciones en C. Para el producto punto y la multiplicación matriz-vector se utilizó loop unrolling. Para la multiplicación matriz-matriz se modificó el orden de los ciclos con el objetivo de mejorar la localidad de memoria.

4. Metodología

Las pruebas fueron realizadas en WSL utilizando Octave, GCC, CBLAS y Gnuplot. Para cada tamaño del problema se ejecutó cada algoritmo siete veces y se calculó el tiempo promedio.

Las mediciones se realizaron utilizando:

- `tic` y `toc` en Octave.
- `gettimeofday` en lenguaje C.
- Scripts Bash para automatizar las pruebas.
- Gnuplot para generar las gráficas.

Los tamaños de prueba fueron seleccionados de manera que los tiempos de ejecución fueran representativos, procurando alcanzar tiempos mayores a 0,01 segundos, tal como solicita el enunciado de la práctica.

5. Implementaciones

5.1. Producto punto

El producto punto se implementó de tres formas principales:

- Operador nativo de Octave.
- Implementación manual en C.
- Implementación optimizada en C mediante loop unrolling.
- Función `cblas_ddot`.

La implementación manual en C utiliza un ciclo simple que acumula el producto de los elementos correspondientes de ambos vectores.

5.2. Multiplicación matriz-vector

La multiplicación matriz-vector se implementó de forma manual mediante dos ciclos anidados. Posteriormente, se generó una versión optimizada que utiliza un acumulador local y loop unrolling en el ciclo interno.

5.3. Multiplicación matriz-matriz

La multiplicación matriz-matriz se implementó inicialmente mediante tres ciclos anidados en el orden clásico i, j, k . Luego, se implementó una versión optimizada cambiando el orden de los ciclos a i, k, j , con el objetivo de mejorar el acceso a memoria en arreglos almacenados en orden row-major.

6. Resultados

6.1. Comparación Octave

6.1.1. Producto punto en Octave

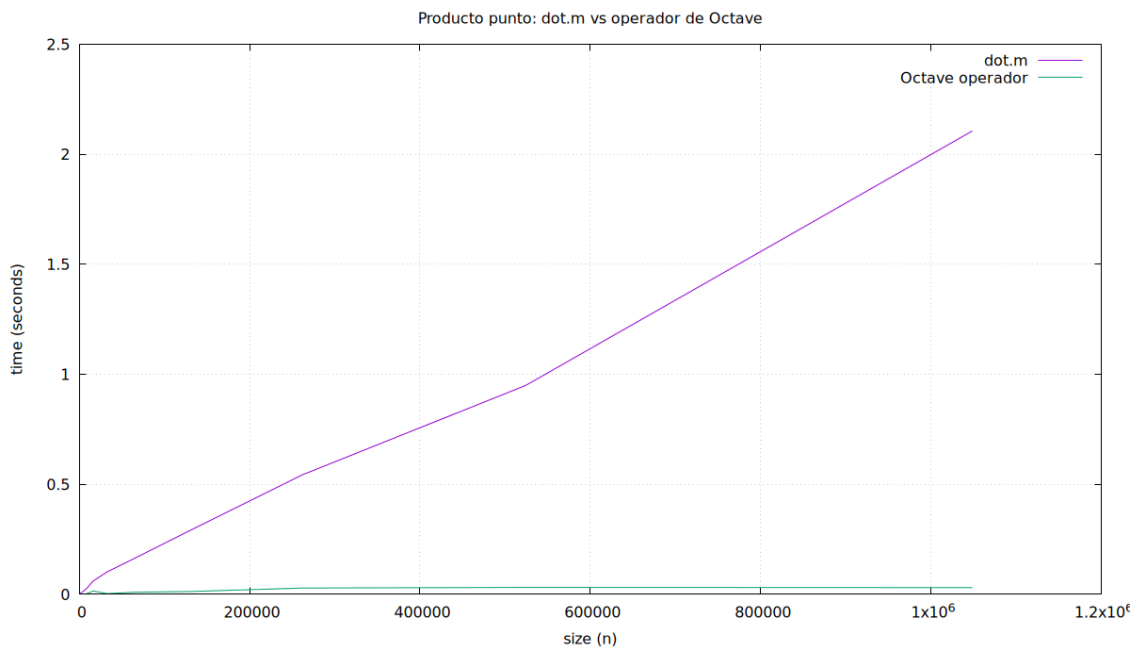


Figura 1: Comparación del producto punto en Octave.

6.1.2. Multiplicación matriz-vector en Octave

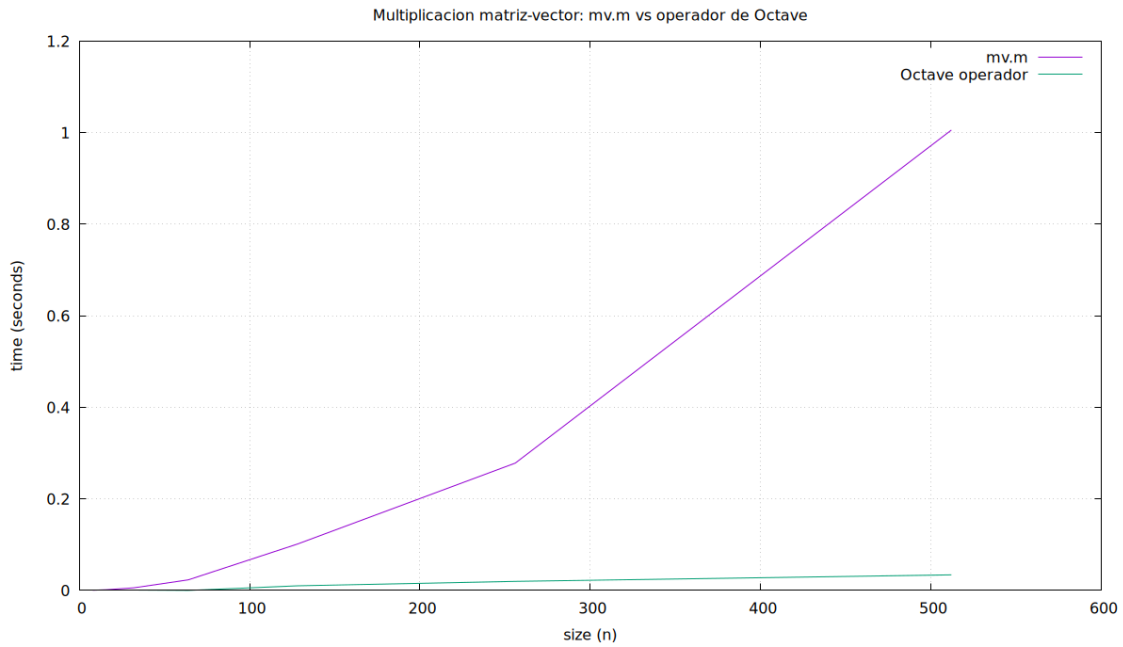


Figura 2: Comparación de multiplicación matriz-vector en Octave.

6.1.3. Multiplicación matriz-matriz en Octave

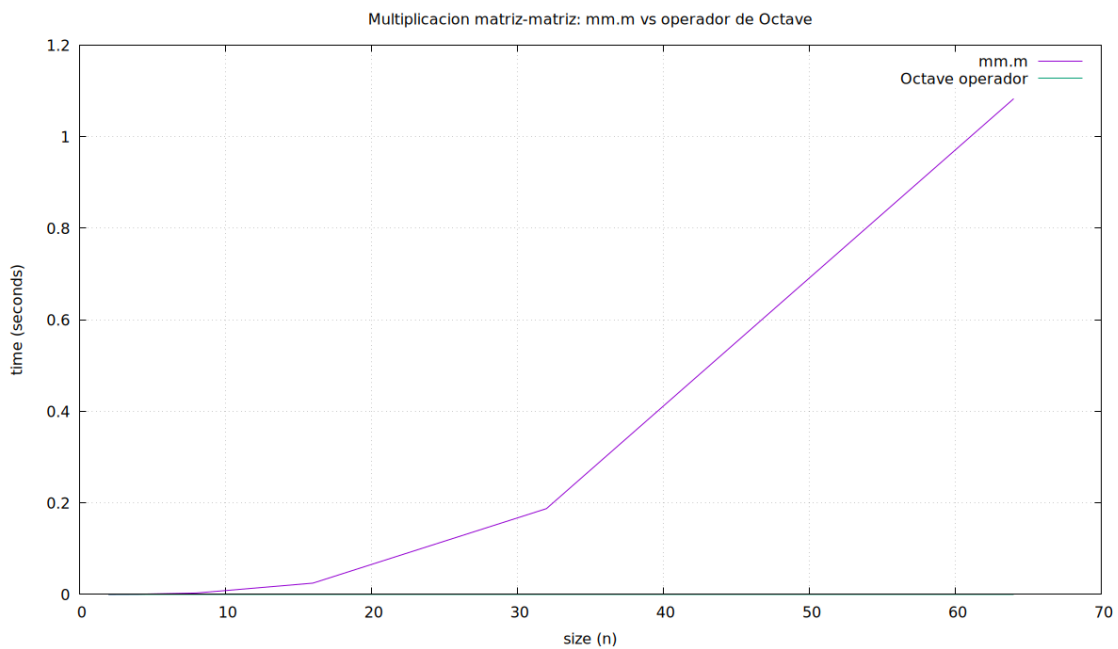


Figura 3: Comparación de multiplicación matriz-matriz en Octave.

6.2. Comparación CBLAS

6.2.1. Producto punto con CBLAS

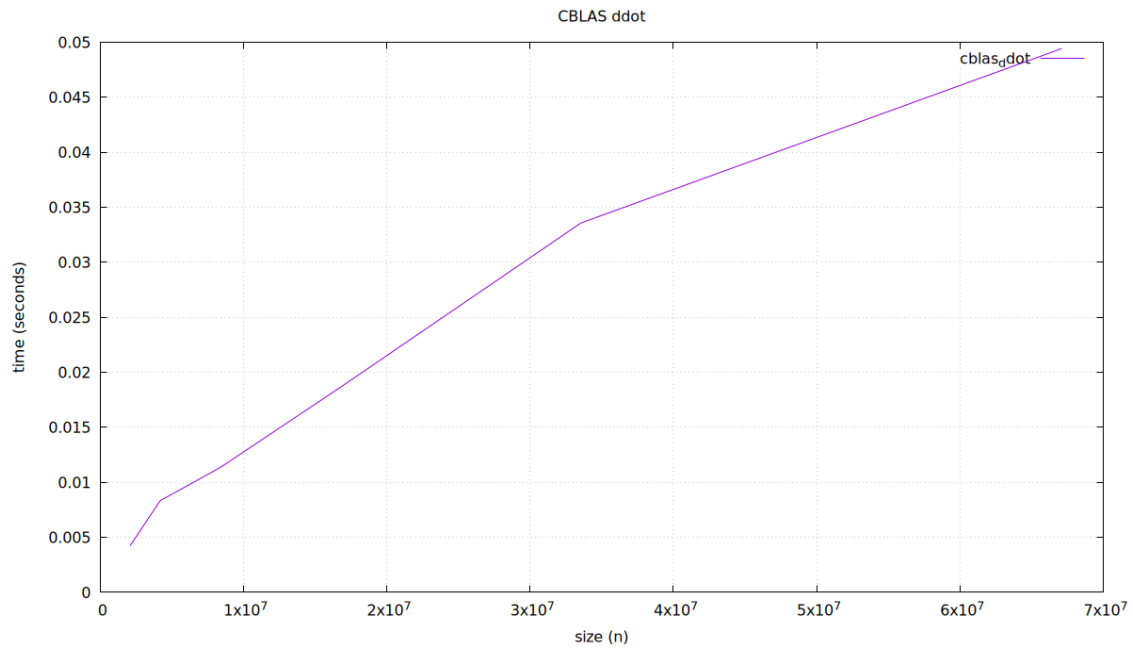


Figura 4: Tiempo de ejecución de `cblas_ddot`.

6.2.2. Multiplicación matriz-vector con CBLAS

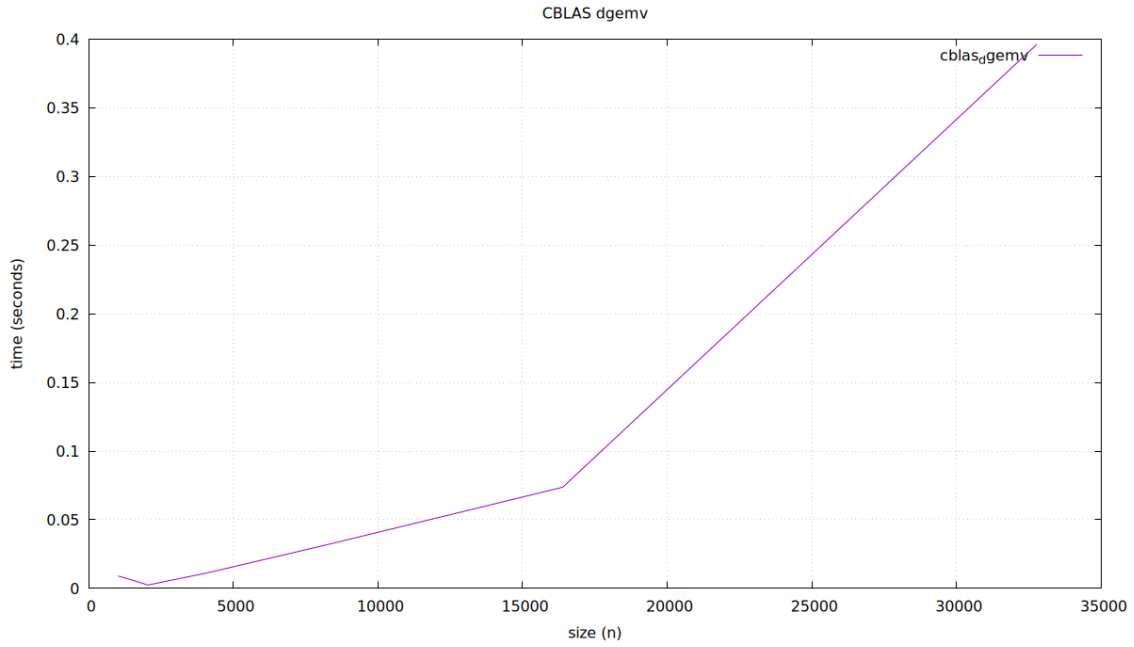


Figura 5: Tiempo de ejecución de `cblas_dgemv`.

6.2.3. Multiplicación matriz-matriz con CBLAS

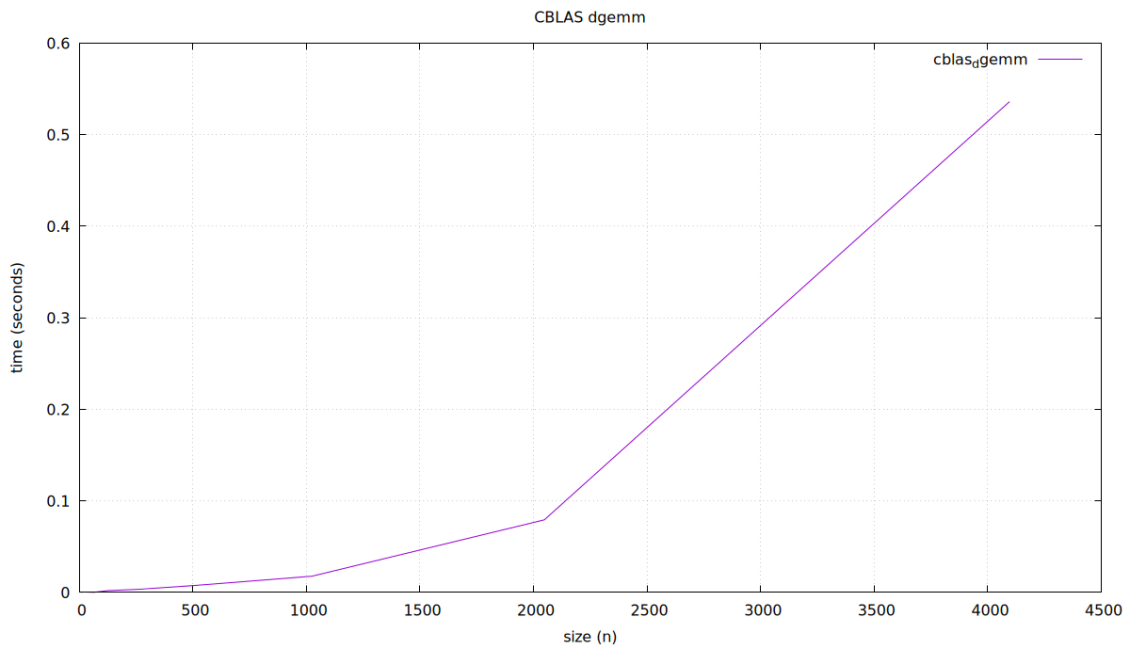


Figura 6: Tiempo de ejecución de `cblas_dgemm`.

6.3. Comparación C manual, C optimizado y CBLAS

6.3.1. Producto punto

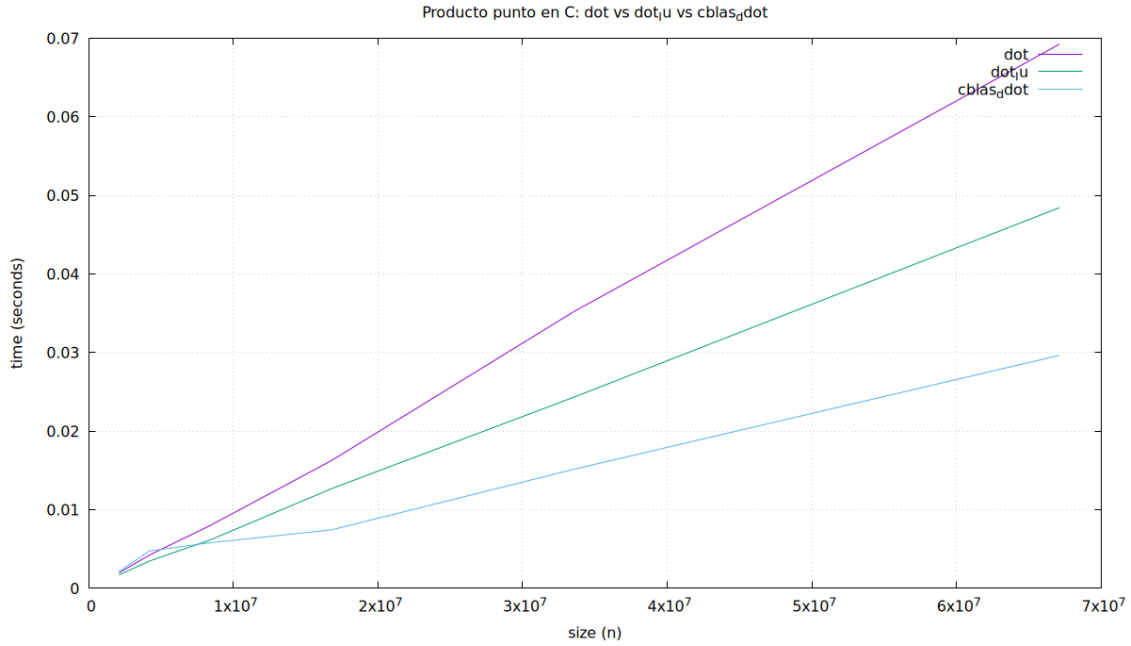


Figura 7: Comparación entre producto punto manual, optimizado y CBLAS.

6.3.2. Multiplicación matriz-vector

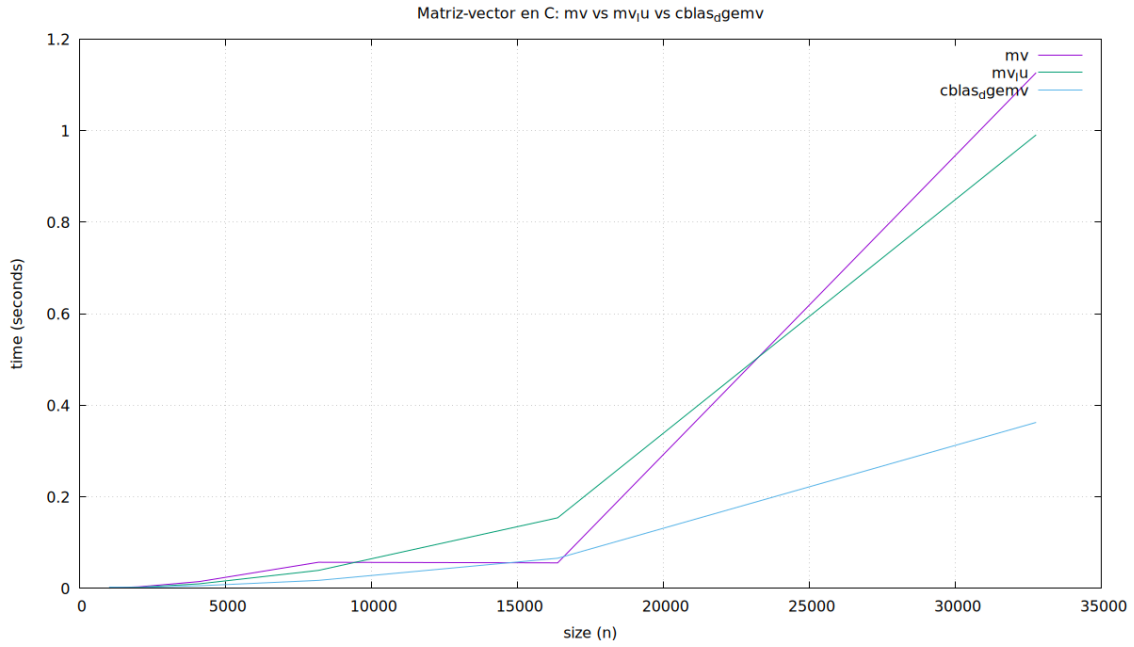


Figura 8: Comparación entre matriz-vector manual, optimizado y CBLAS.

6.3.3. Multiplicación matriz-matriz

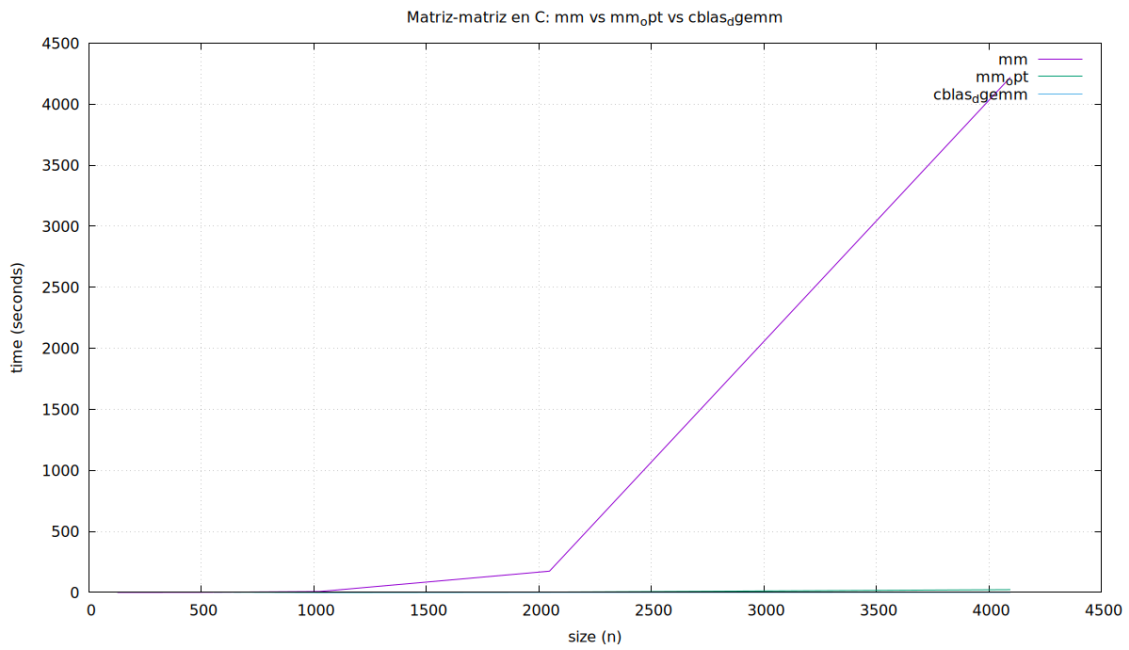


Figura 9: Comparación entre matriz-matriz manual, optimizada y CBLAS.

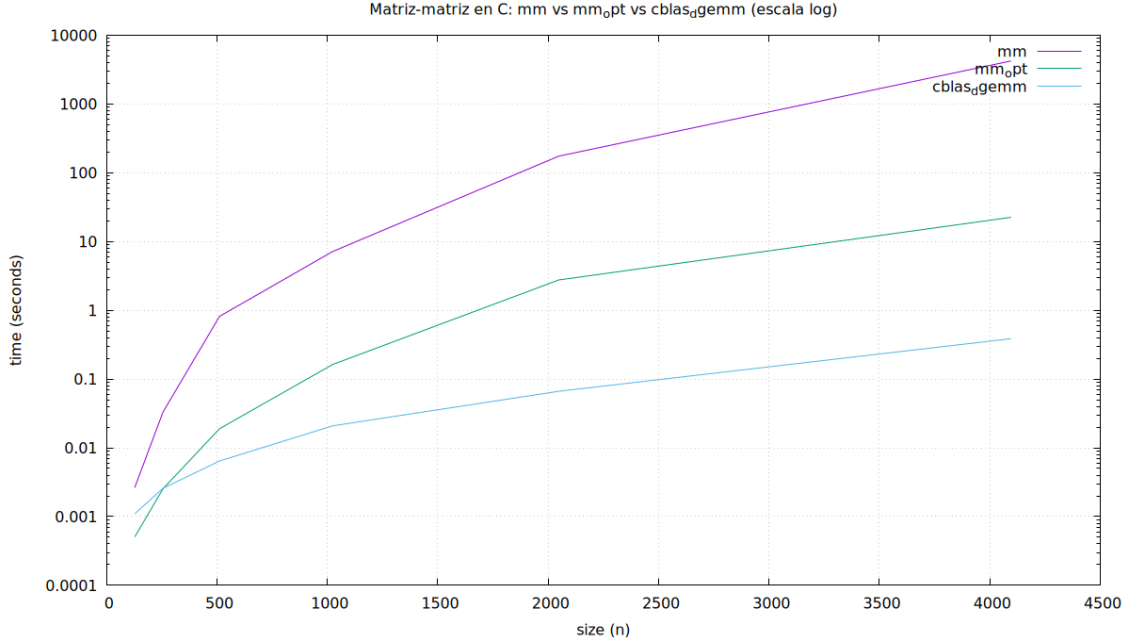


Figura 10: Comparación matriz-matriz en escala logarítmica.

7. Análisis de resultados

7.1. Producto punto

En el producto punto se observa un comportamiento aproximadamente lineal, coherente con su complejidad $O(n)$. La versión optimizada mediante loop unrolling logra reducir el tiempo de ejecución respecto a la versión manual. Sin embargo, la función `cblas_ddot` presenta mejores resultados para tamaños grandes, debido a que corresponde a una rutina especializada y optimizada.

7.2. Multiplicación matriz-vector

En la multiplicación matriz-vector se observa un crecimiento más pronunciado que en el producto punto, lo cual es coherente con su complejidad $O(n^2)$. La versión optimizada logra mejorar el rendimiento en varios tamaños, aunque se observaron algunas mediciones irregulares en tamaños intermedios. Estas variaciones pueden asociarse a ruido del sistema, carga de CPU o efectos de memoria.